

COMP 4423: IMAGE CLASSIFICATION FUNDAMENTAL

Easy Computer Vision

Xiaoyong Wei (魏驍勇)
x1wei@polyu.edu.hk

 THE HONG KONG
POLYTECHNIC UNIVERSITY
香港理工大學

Opening Minds • Shaping the Future
啟迪思維 • 成就未來



New Toy

Testing ..

Rock

```
...n_test.py 87
ures= 300
eat= 24.7445950
954
(10)
= 2
ures= 300
eat= 24.4998880
27
(10)
= 2
ures= 300
eat= 24.7053649
932
(10)
= 2
ures= 300
eat= 24.5307696
812
(10)
= 2
ures= 300
eat= 25.1461625
038
(10)
= 2
ures= 300
eat= 26.2269585
0365
(10)
= 2
ures= 300
eat= 25.9408358
177
(10)
= 2
ures= 300
```

```
144
145 if len(results) == 0:
```

master* 0 3

Ln 138, Col 39 Spaces: 4 UTF-8 LF Python 3.10.8 64-bit



Outline

- Classification
- Supervised learning
- K nearest neighbors (k-NN)
- Bayesian classifiers
- Support vector machines (SVM)
- Rock-Paper-Scissors

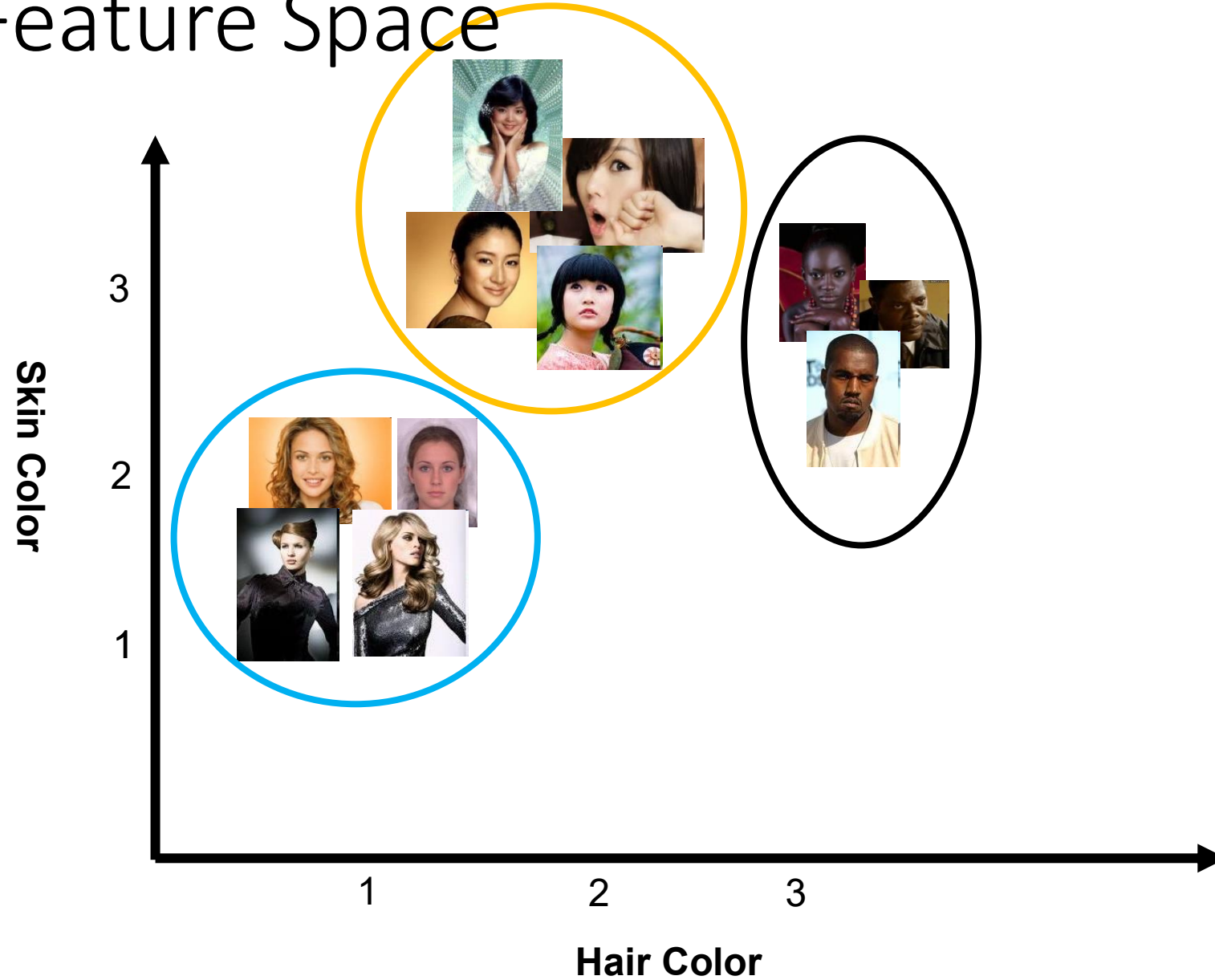


How do you group them?





Feature Space





Clustering is **unsupervised learning** which means we (human) don't have to tell the computers what each group looks like. It's data-driven without using human knowledge (supervision).

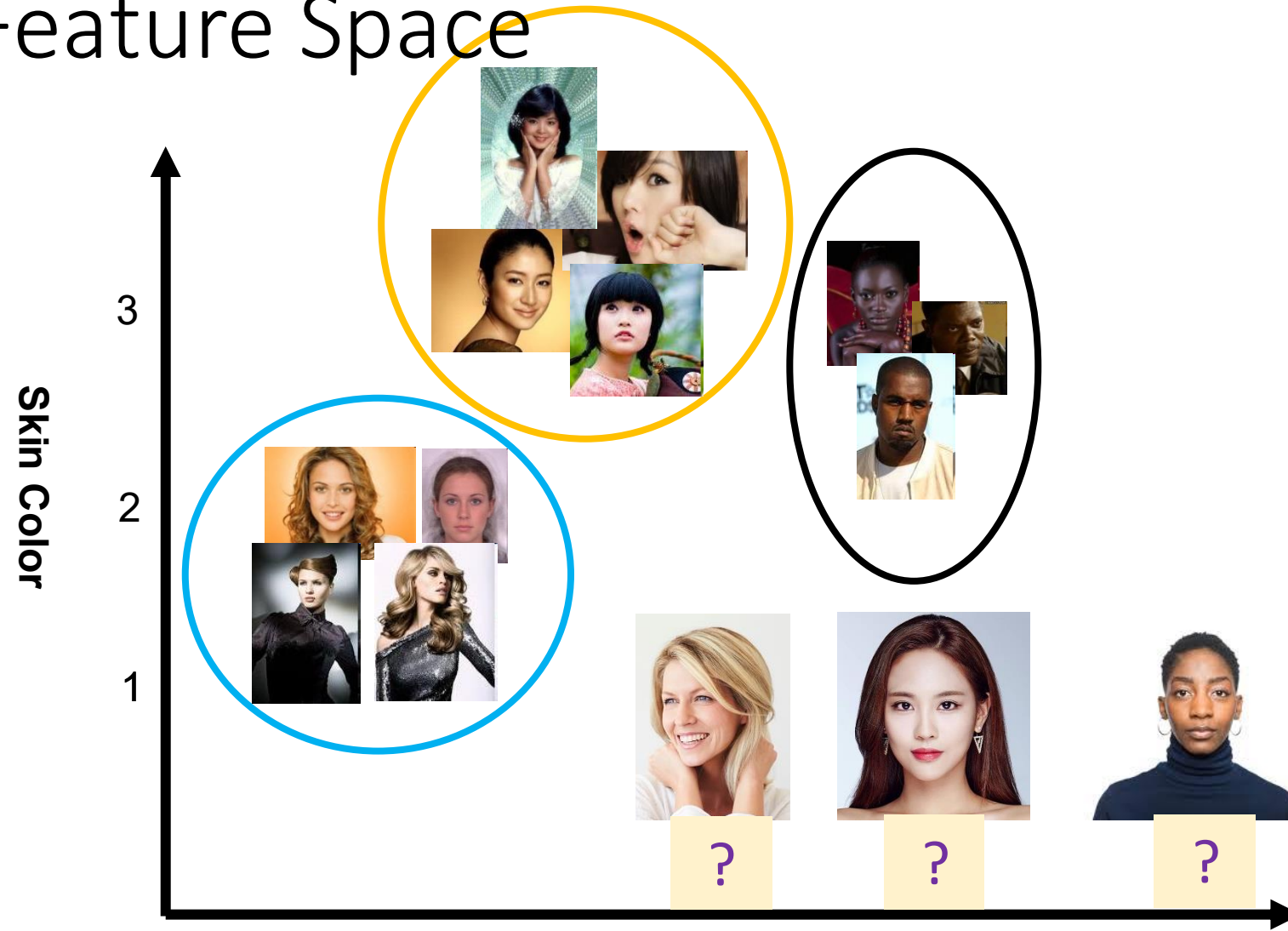


Sounds good?

But ...



Feature Space



What if we have new examples unseen?



Feature Space

What if the features selected are not representative enough or consistent with our understanding?

skin Color

1

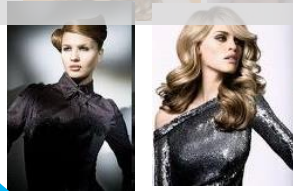
2

1

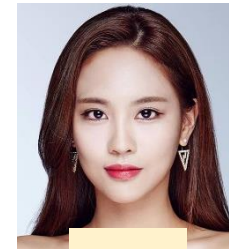
2

3

Hair Color



age
?



age
?



age
?



We can **tell** the computers about **our**
understanding on the subjects by
giving **labels**.



Training Examples (Seen)

	Hair Color (H)	Skin Color (S)	Class Label (L)
	Yellow (2)	White (1)	W
	Black (3)	Yellow (2)	A
	Black (3)	Yellow (2)	A
	Yellow (2)	White (1)	W
	Black (3)	Black (3)	B



Testing Examples (Unseen)

	Hair Color (H)	Skin Color (S)	Class Label (L)
	2.2	0.8	?
	3.2	1.9	?
	3.1	2.2	?
	2.4	1.3	?
	3.1	2.9	?



We can **tell** the computers about **our**
understanding on the subjects by
giving **labels**.



Training Examples (Seen)

	Hair Color (H)	Skin Color (S)	Class Label (L)
	Yellow (2)	White (1)	W
	Black (3)	Yellow (2)	A
	Black (3)	Yellow (2)	A
	Yellow (2)	White (1)	W
	Black (3)	Black (3)	B



Testing Examples (Unseen)

	Hair Color (H)	Skin Color (S)	Class Label (L)
	2.2	0.8	?
	3.2	1.9	?
	3.1	2.2	?
	2.4	1.3	?
	3.1	2.9	?



Classification: to predict the labels of the testing (unseen) examples based on the knowledge learned from the training (seen) examples



Classification: to predict the labels of the testing (unseen) examples based on the knowledge learned from the training (seen) examples

We are training the computers and the processing is called **training**.
The computers are learning. This is what the term “**machine learning**” is referring to.
Our participation in learning makes it
supervised learning.

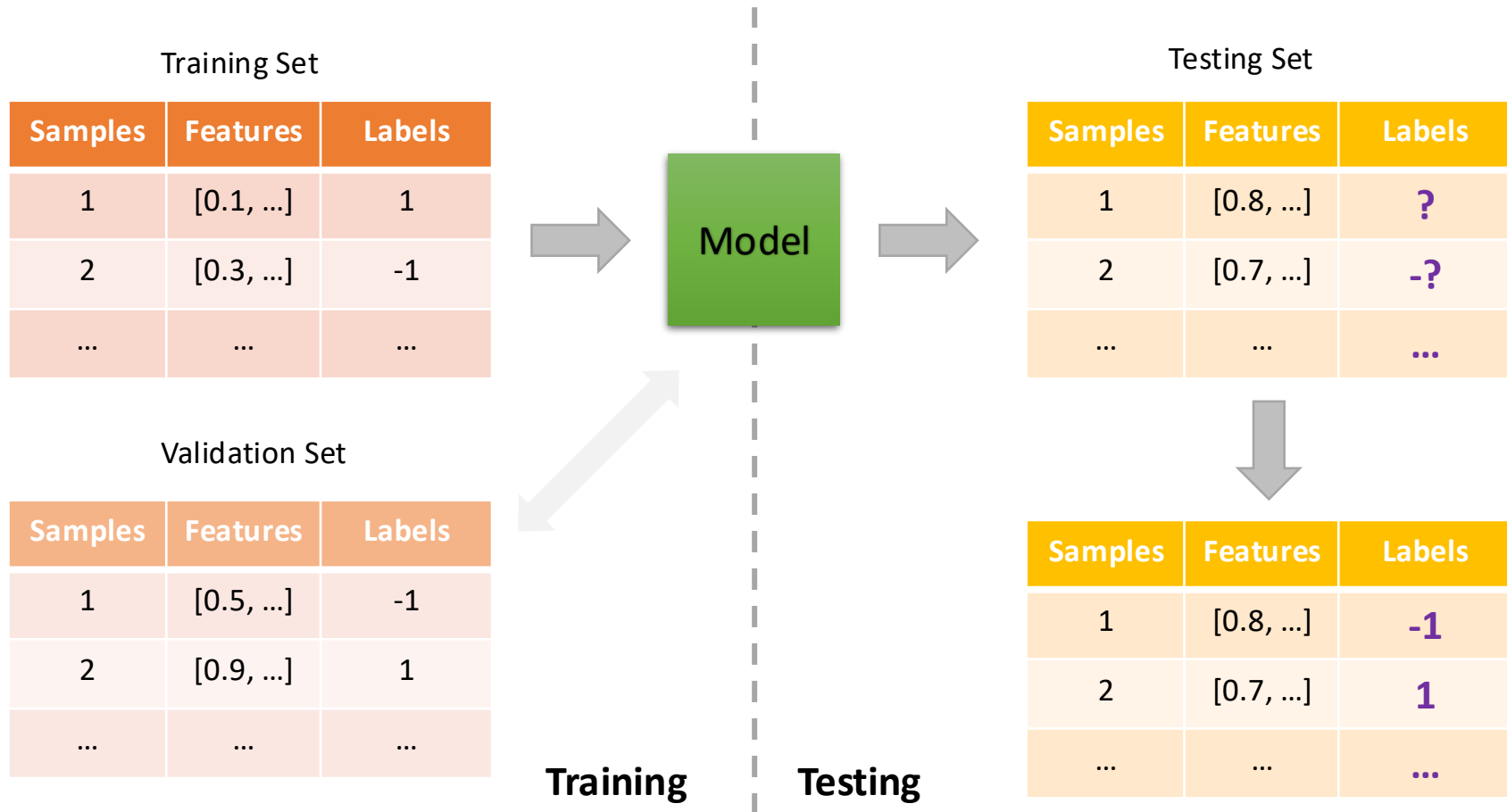


Classification: to predict the labels of the testing (unseen) examples based on the knowledge learned from the training (seen) examples

The result is the machine's understanding of the knowledge. We call it a **model** sometimes.



Classification – Supervised Learning





Then how?



Instance-based Learning



PDA



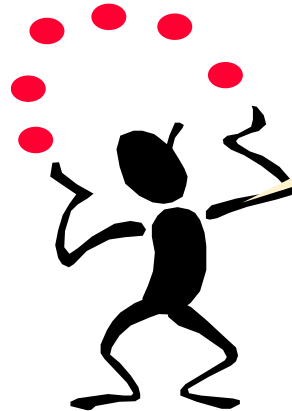
Cellphone



Desktop PC



Laptop



It looks like the
laptop I see last
time.





How about we use image retrieval to find the most similar ones from the seen examples for reference?



kNN Classifiers



kNN Classifier

> NN: nearest neighbors

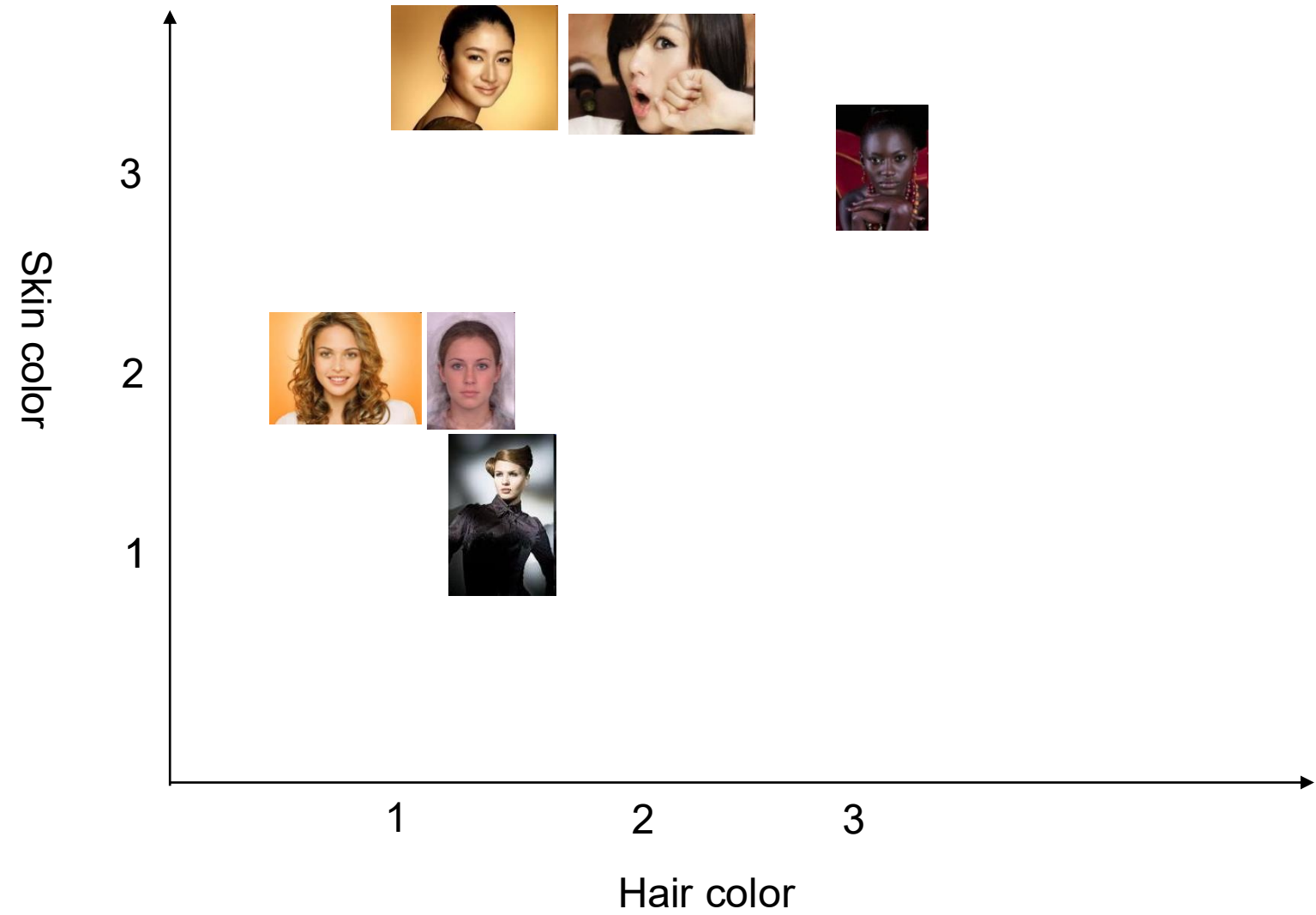
> k: number of nearest neighbors

> Idea

- When $k=1$: assign the unseen with the label of its nearest neighbor
- 近朱者赤, 近墨者黑 (If you lie down with dogs, you will get up with fleas)

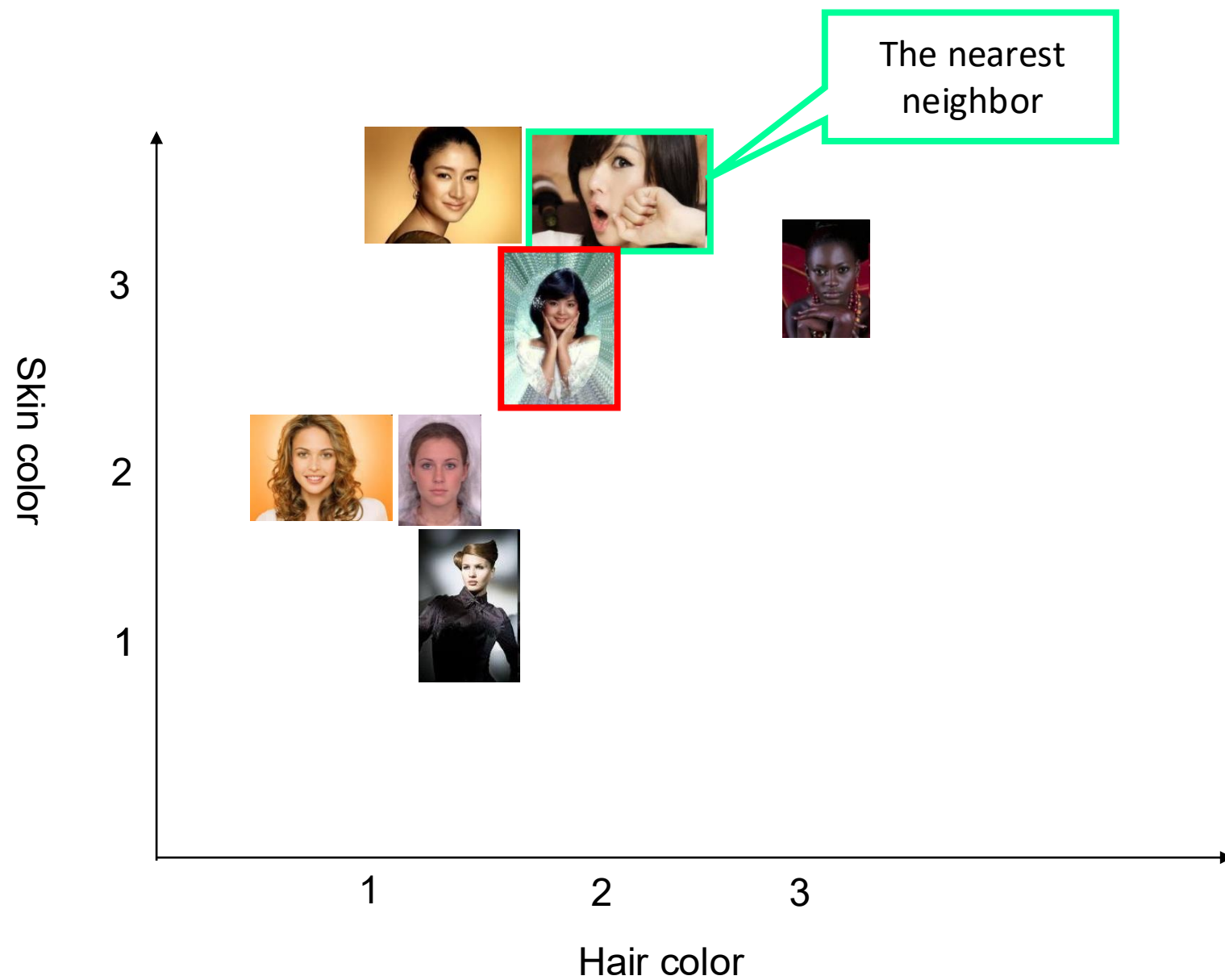


$k=1$





$k=1$





kNN Classifier

> NN: nearest neighbors

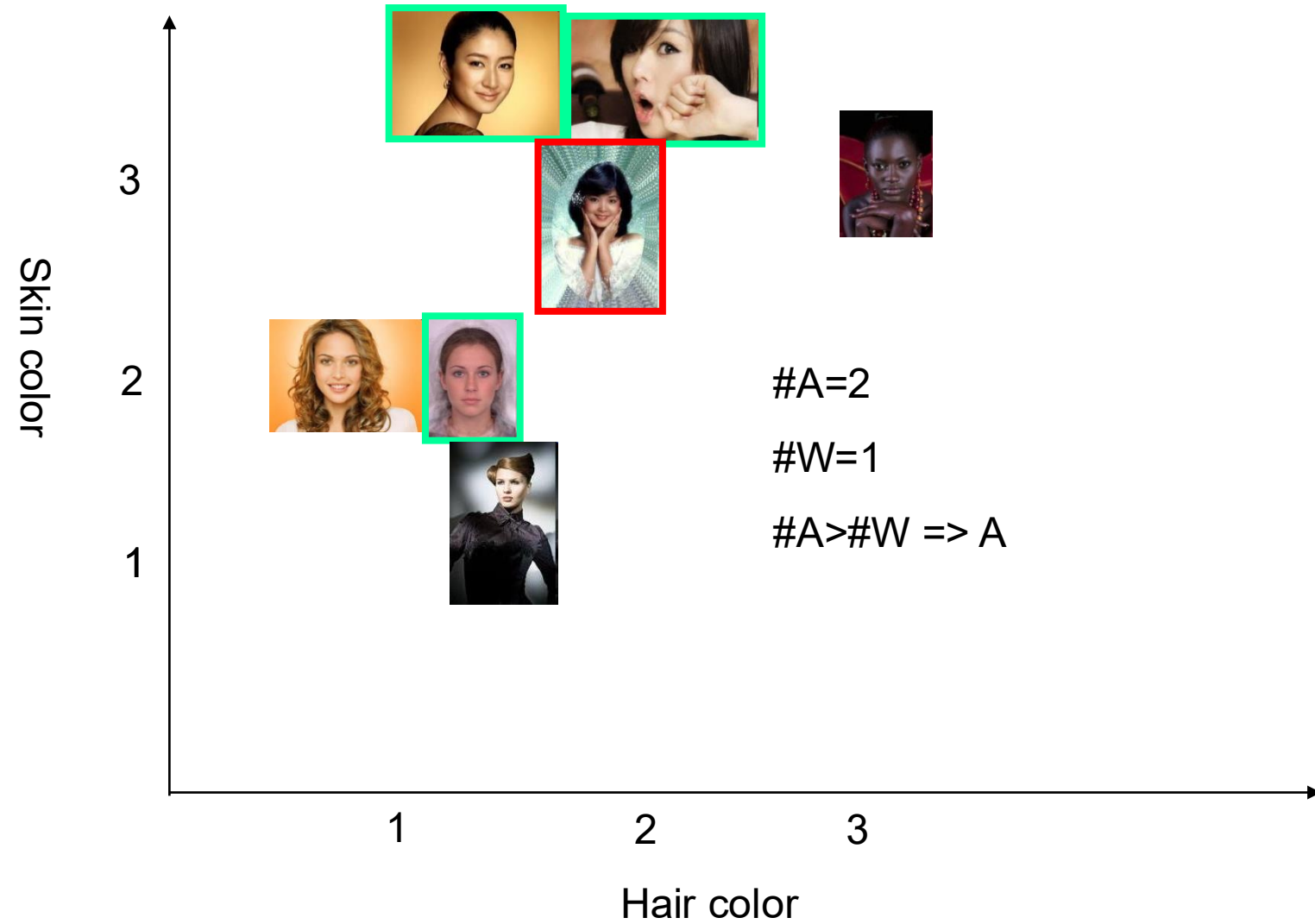
> k: number of nearest neighbors

> Idea

- $k=1$: assign the unseen with the label of its nearest neighbor
- $k>1$: **assign the dominating label among these of the k nearest neighbors**



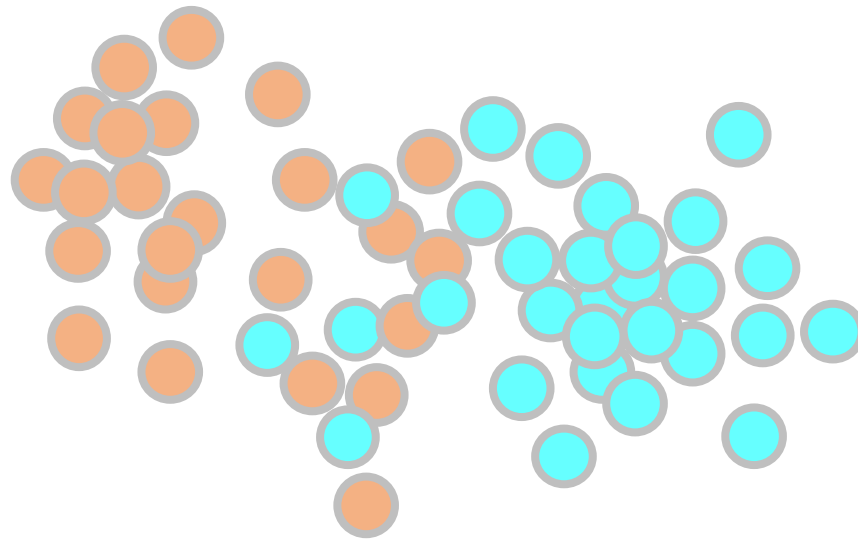
$k=3$





It's straightforward. But so far, we picked the simplest case (classes are well separated) for illustration purpose.

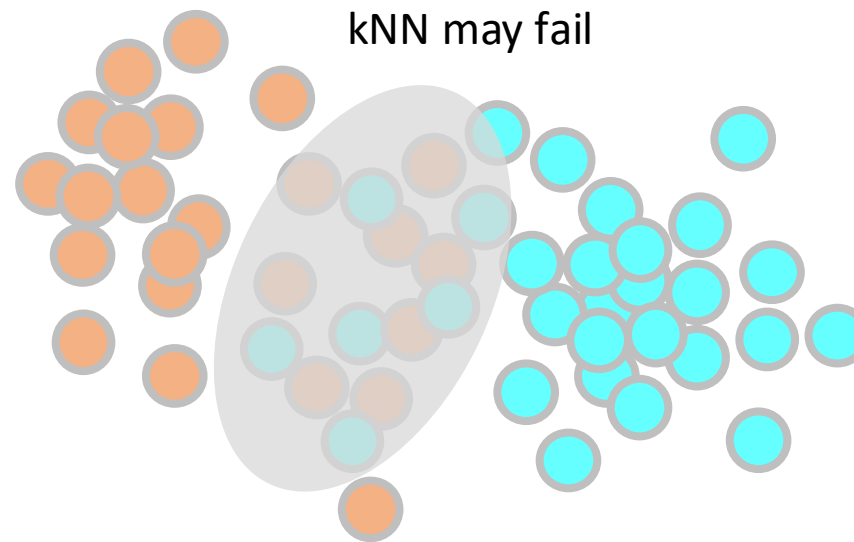
In a more general sense, this is what we're going to have.





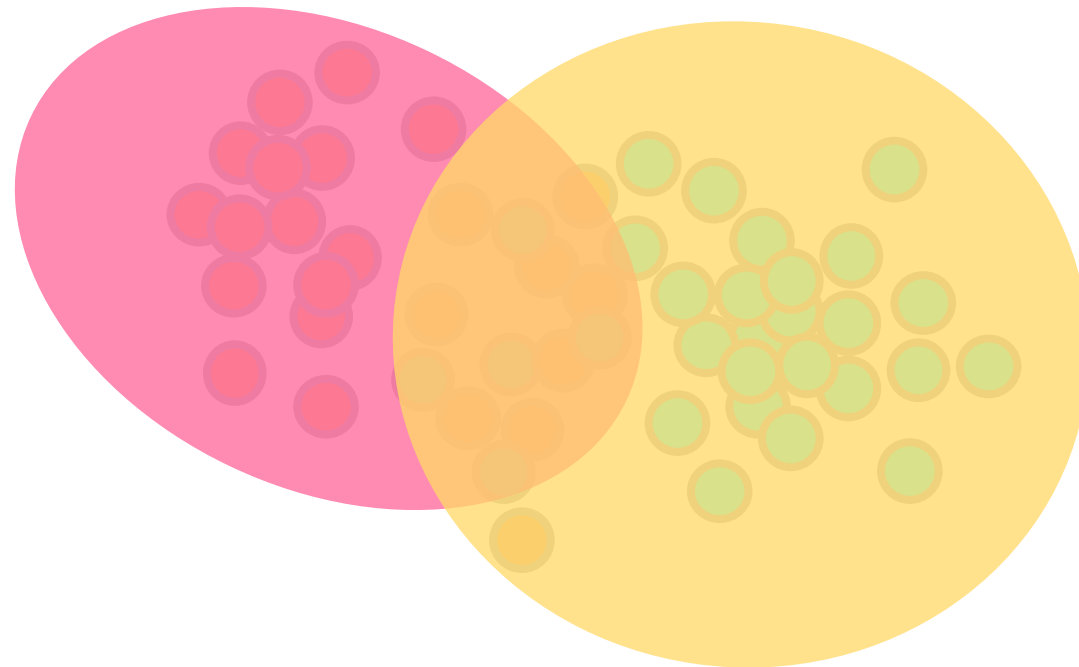
It's straightforward. But so far, we picked the simplest case (classes are well separated) for illustration purpose.

In a more general sense, this is what we're going to have.





Can we teach the computer to draw a “circle”
for each of the class and evaluate the
membership of an example by measuring
how much it falls into “circles”?



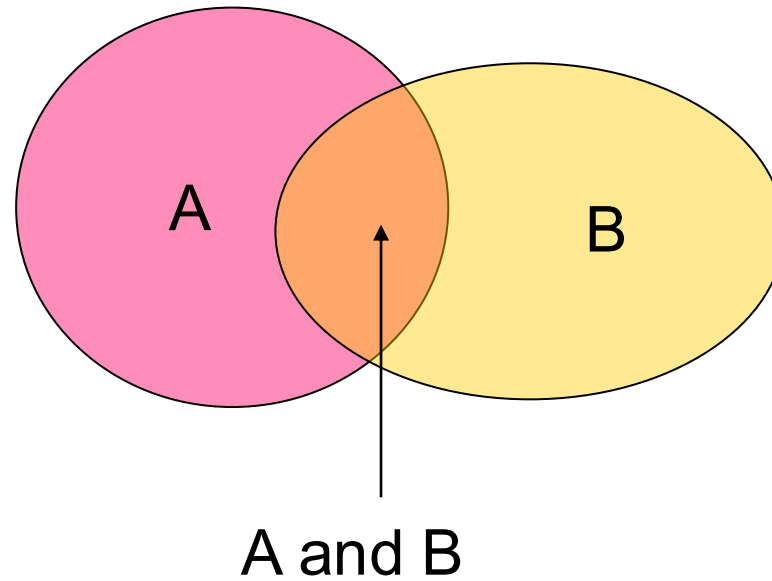


Bayesian Classifiers



Bayesian Classifiers

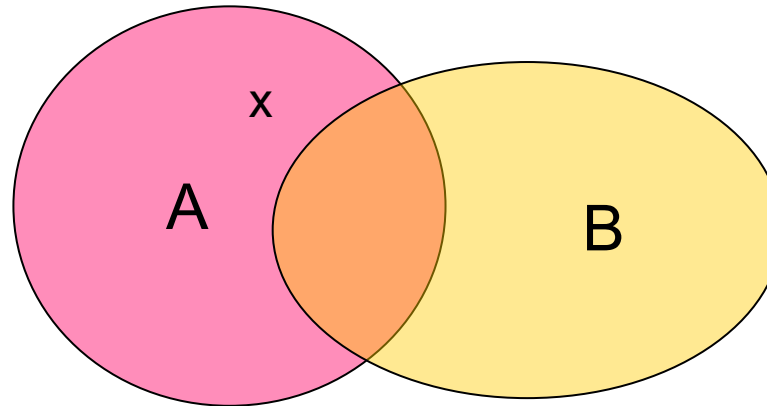
- Classes A and B as two sets





Bayesian Classifiers

- Classes A and B as two sets
 - $P(A|x)$: the probability of A is observed when seeing an x
 - $P(B|x)$: the probability of B is observed when seeing an x





Bayesian Classifiers

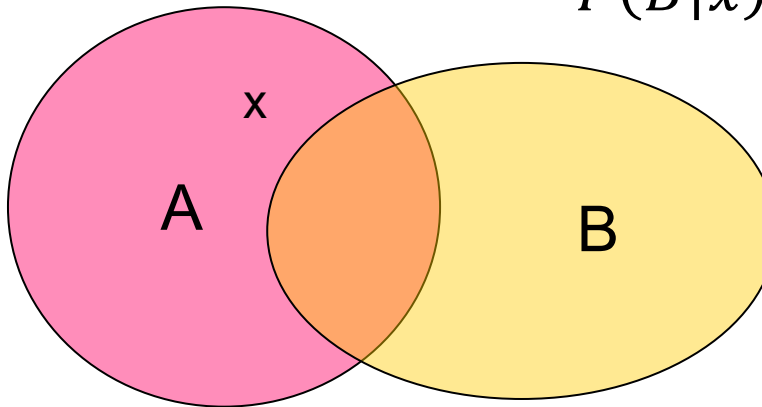
- Classes A and B as two sets

- $P(A|x) \propto P(x|A)P(A)$

- $P(B|x) \propto P(x|B)P(B)$

$$P(A|x) = \frac{P(x|A)P(A)}{P(x)}$$

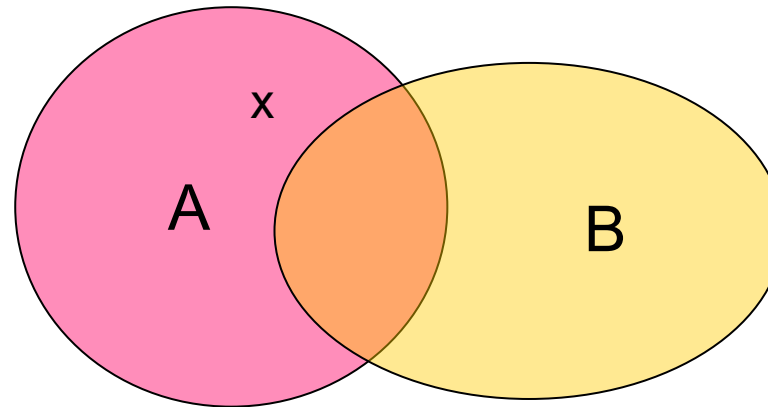
$$P(B|x) = \frac{P(x|B)P(B)}{P(x)}$$





Bayesian Classifiers

- Classes A and B as two sets
 - $P(A|x) \propto P(x|A)P(A)$
 - $P(B|x) \propto P(x|B)P(B)$



$$P(A) = \#A / (\#A + \#B)$$

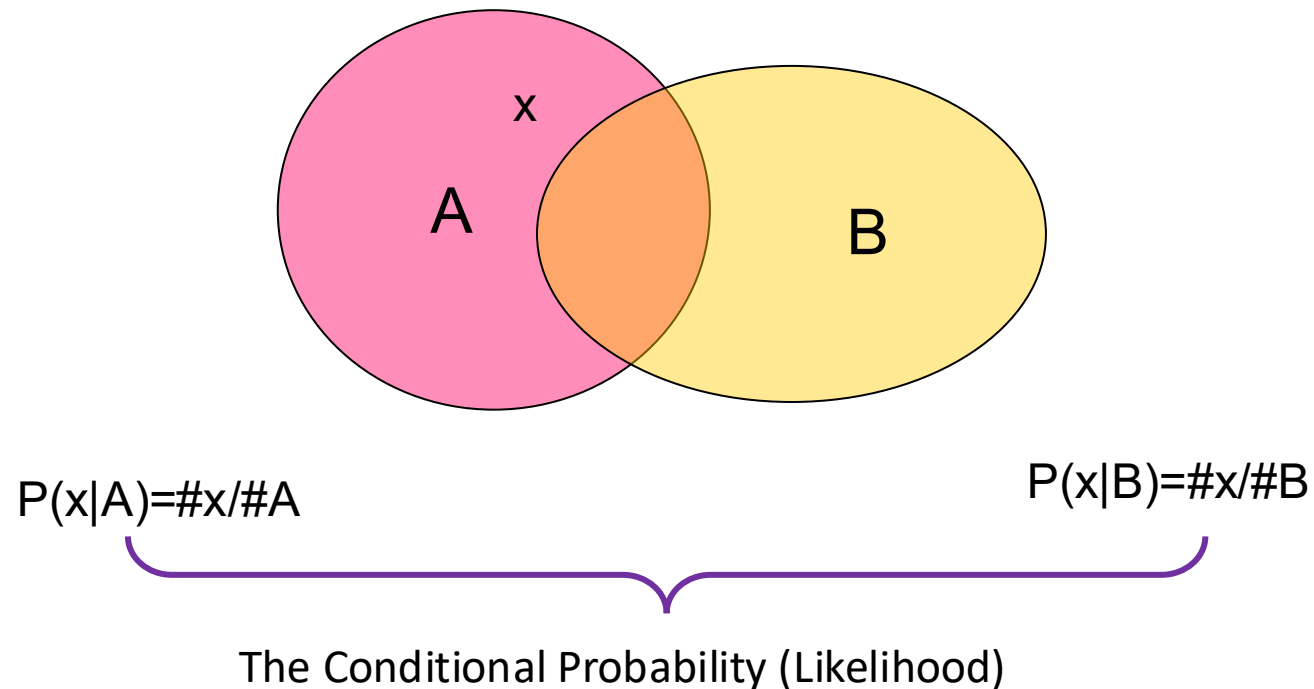
$$P(B) = \#B / (\#A + \#B)$$

The Prior Probability



Bayesian Classifiers

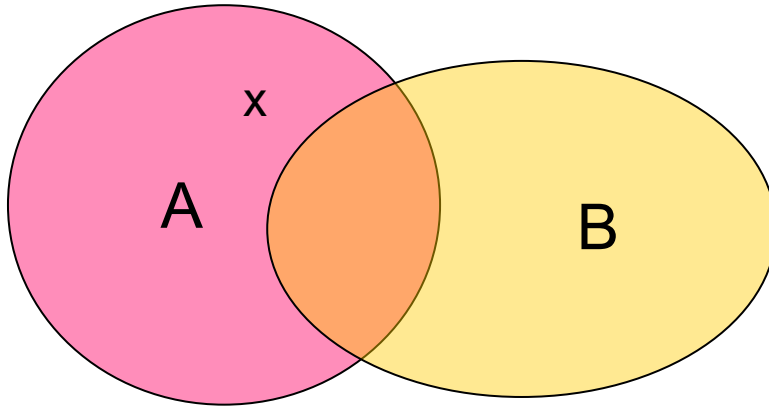
- Classes A and B as two sets
 - $P(A|x) \propto P(x|A)P(A)$
 - $P(B|x) \propto P(x|B)P(B)$





Bayesian Classifiers

- Classes A and B as two sets
 - $P(A|x) \propto P(x|A)P(A)$
 - $P(B|x) \propto P(x|B)P(B)$

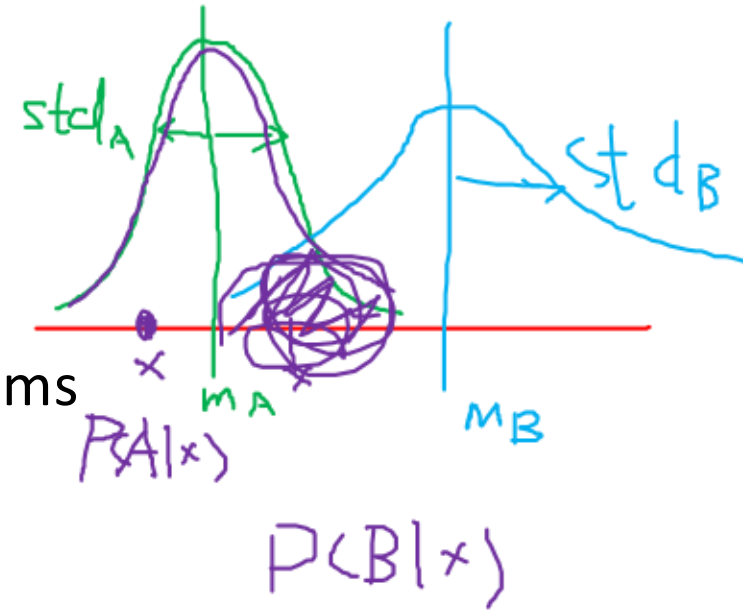


Decision function: x is A if $P(A|x) > P(B|x)$,
 B otherwise



Naive Bayesian

- Advantages:
 - Fast
 - Extendable to multi-class problems
 - Requires less training examples
 - Works well for categorical data
- Disadvantages:
 - **Features are assumed to be independent to each other** (not true in real-world applications)
 - Zero-frequency problem



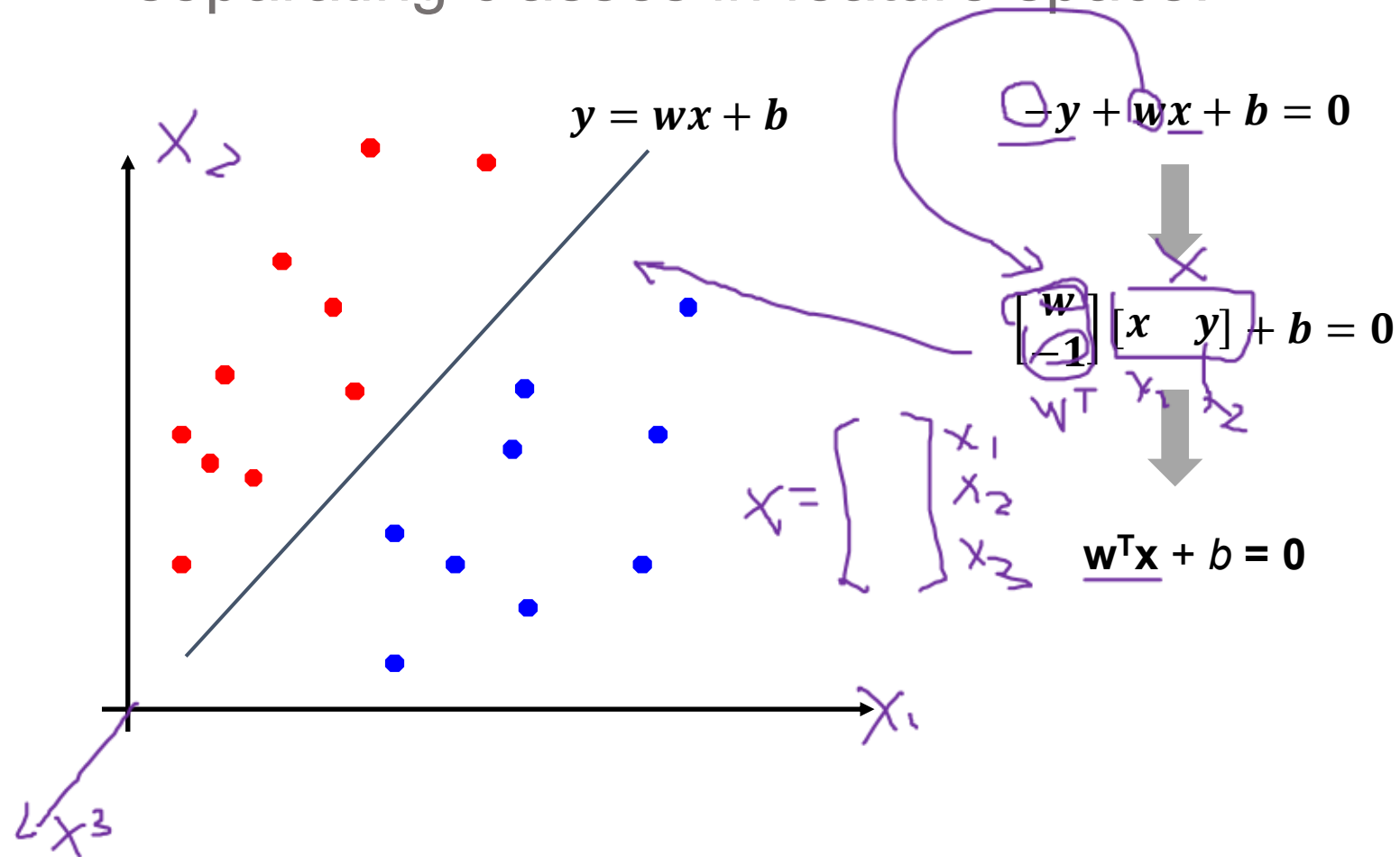


Support Vector Machines



Linear Separators

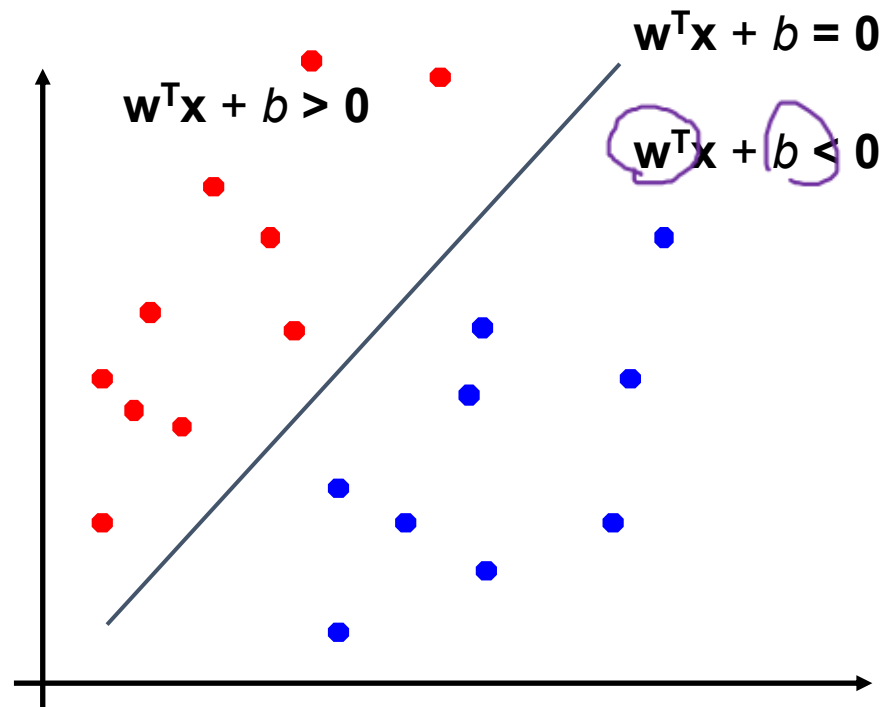
> Binary classification can be viewed as the task of separating classes in feature space:





Linear Separators

> Binary classification can be viewed as the task of separating classes in feature space:



$$\begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \begin{bmatrix} x_1 & x_2 \end{bmatrix} + b$$

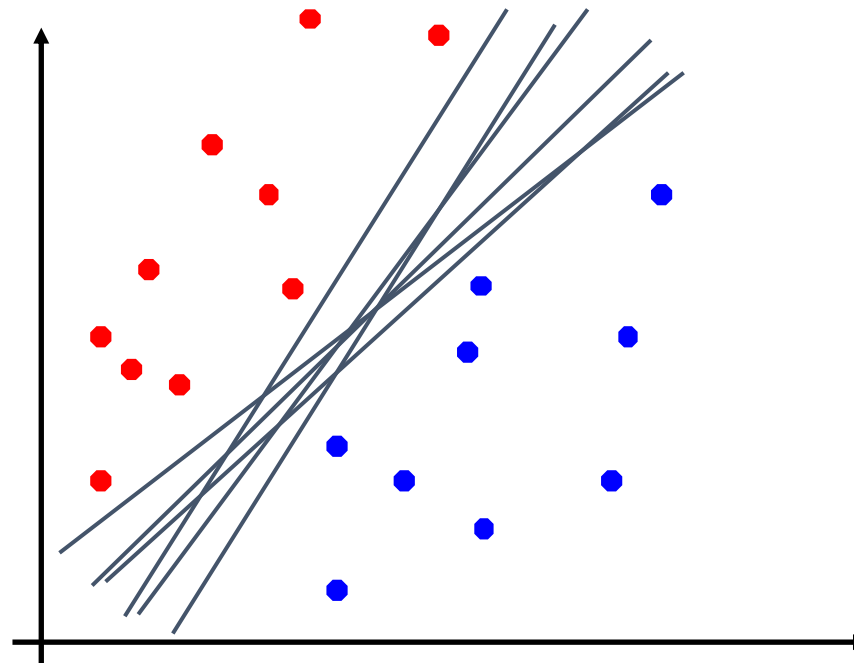
$$f(\mathbf{x}) = \text{sign}(w^T \mathbf{x} + b)$$

$$\begin{bmatrix} w \\ \begin{bmatrix} w_1 \\ w_2 \end{bmatrix} \\ b \end{bmatrix} \begin{bmatrix} x_1 & x_2 & 1 \end{bmatrix}$$



Linear Separators

> Binary classification can be viewed as the task of separating classes in feature space:

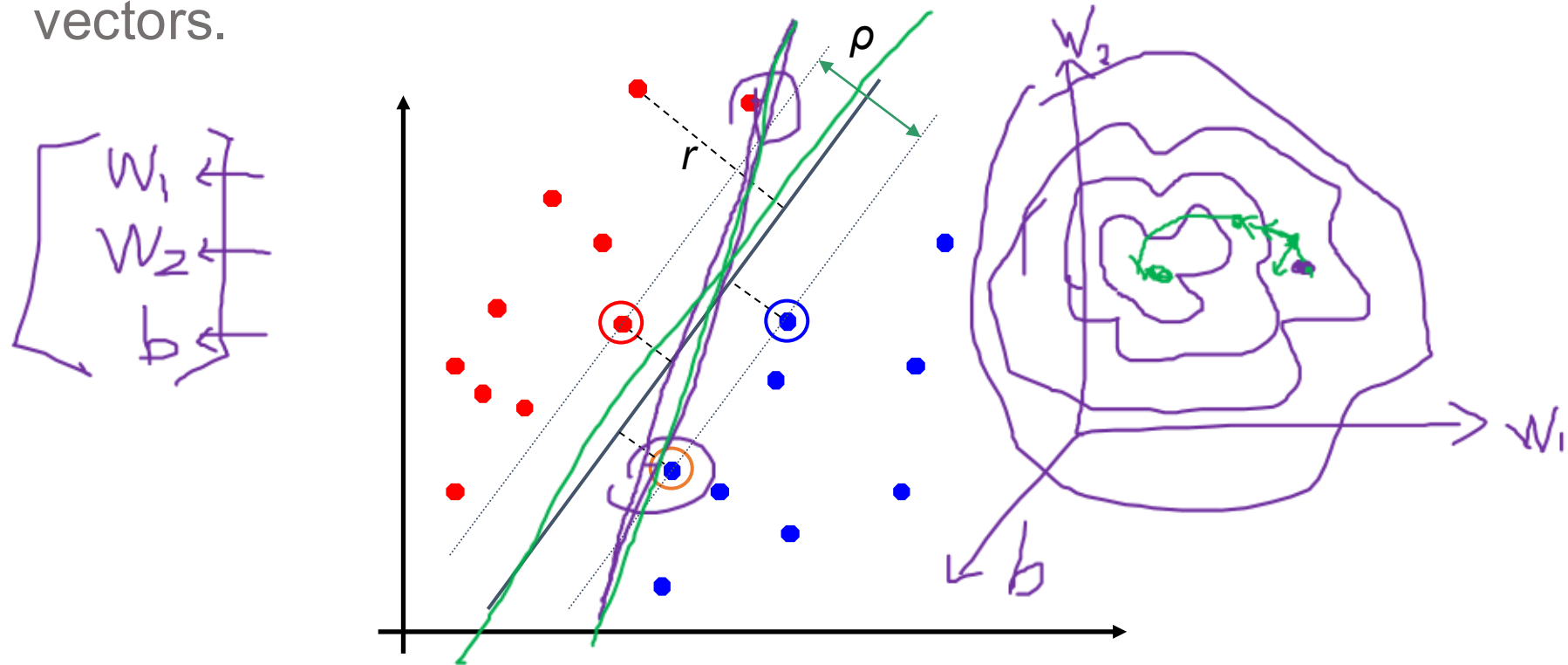


> Which one is the best?



Margin

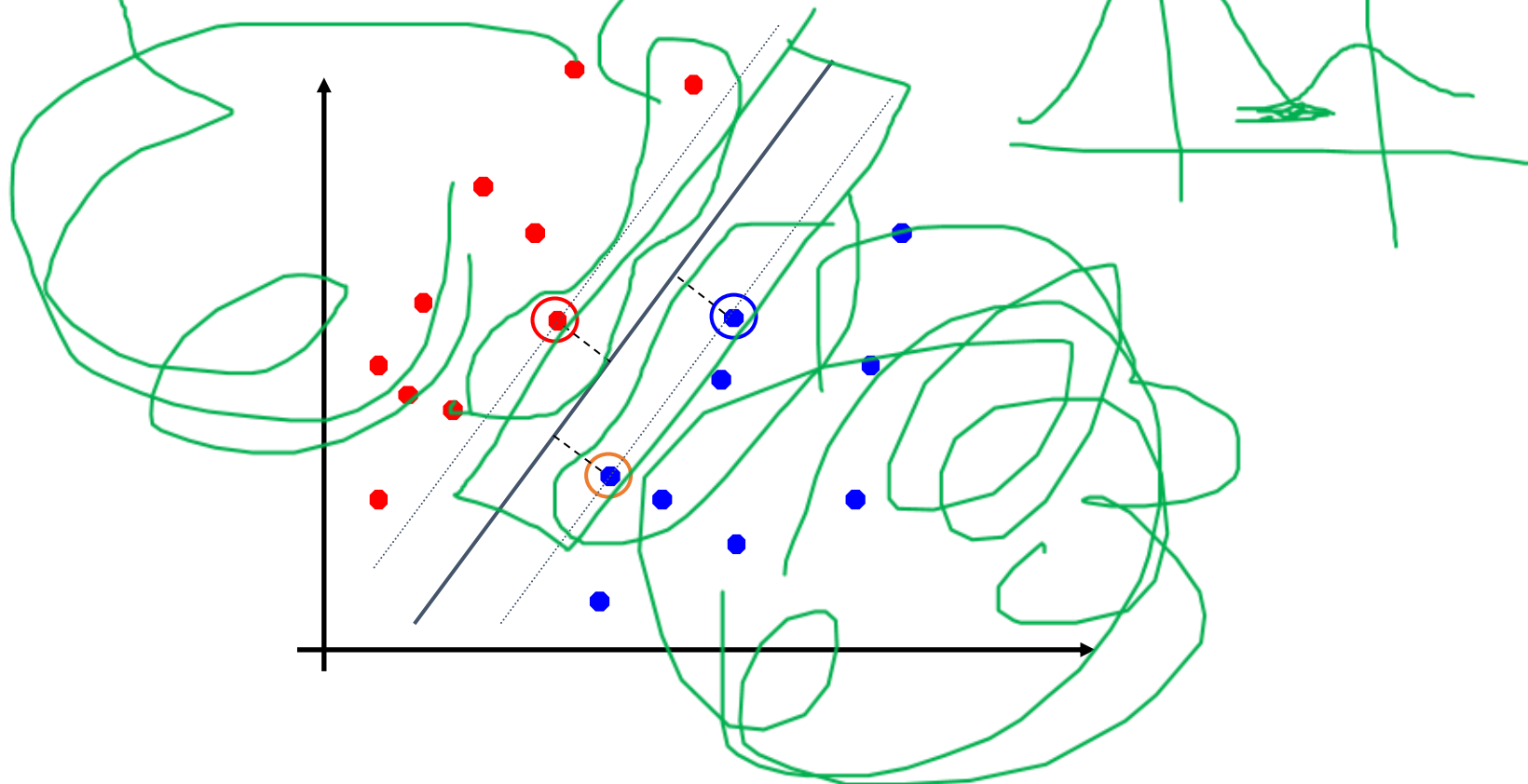
- > Distance from example $\mathbf{x}_i$ to the separator is $r = \frac{\mathbf{w}^T \mathbf{x}_i + b}{\|\mathbf{w}\|}$
- > Examples closest to the hyperplane are **support vectors**.
- > **Margin** ρ of the separator is the distance between support vectors.





Maximum Margin Classification

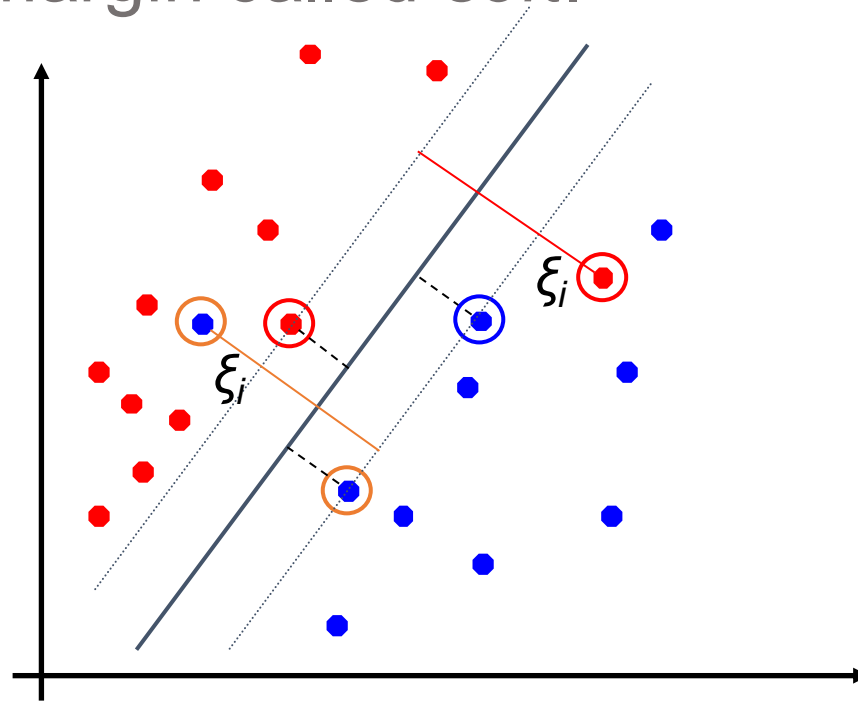
- > Maximizing the margin is good according to intuition and PAC theory (Probably Approximately Correct).
- > Implies that only support vectors matter; other training examples are ignorable.





Soft Margin Classification

- > What if the training set is not linearly separable?
- > *Slack variables* ξ_i can be added to allow misclassification of difficult or noisy examples, resulting margin called *soft*.





Linear SVMs: Overview

- > The classifier is a *separating hyperplane*
- > Most “important” training points are support vectors; they define the hyperplane.
- > Quadratic optimization algorithms can identify which training points $\mathbf{x}_i$ are support vectors with non-zero Lagrangian multipliers α_i .
- > Both in the dual formulation of the problem and in the solution training points appear only inside inner products:

Find $\alpha_1 \dots \alpha_N$ such that

$\mathbf{Q}(\boldsymbol{\alpha}) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

(1) $\sum \alpha_i y_i = 0$

(2) $0 \leq \alpha_i \leq C$ for all α_i

$$f(\mathbf{x}) = \sum \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$



Linear SVMs: Overview

- > The classifier is a *separating hyperplane*
- > Most “important” training points are support vectors; they define the hyperplane.
- > Quadratic optimization algorithms can identify which training points $\mathbf{x}_i$ are support vectors with non-zero Lagrangian multipliers α_i .
- > Both in the dual formulation of the problem and in the solution training points appear only inside inner products:

Find $\alpha_1 \dots \alpha_N$ such that

$\mathbf{Q}(\boldsymbol{\alpha}) = \sum \alpha_i - \frac{1}{2} \sum \sum \alpha_i \alpha_j y_i y_j \mathbf{x}_i^T \mathbf{x}_j$ is maximized and

(1) $\sum \alpha_i y_i = 0$

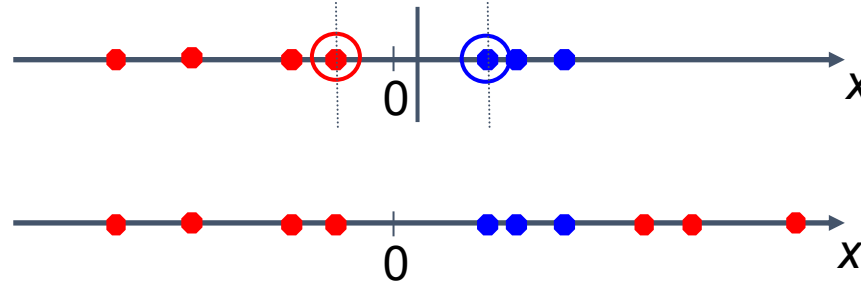
(2) $0 \leq \alpha_i \leq C$ for all α_i

$$f(\mathbf{x}) = \sum \alpha_i y_i \mathbf{x}_i^T \mathbf{x} + b$$



Non-linear SVMs

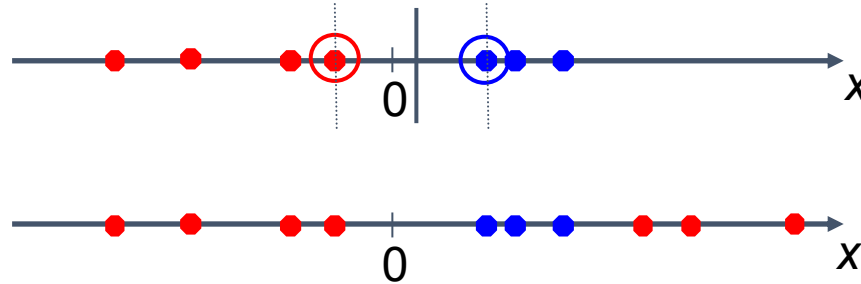
> Datasets that are linearly separable with some noise work out great:



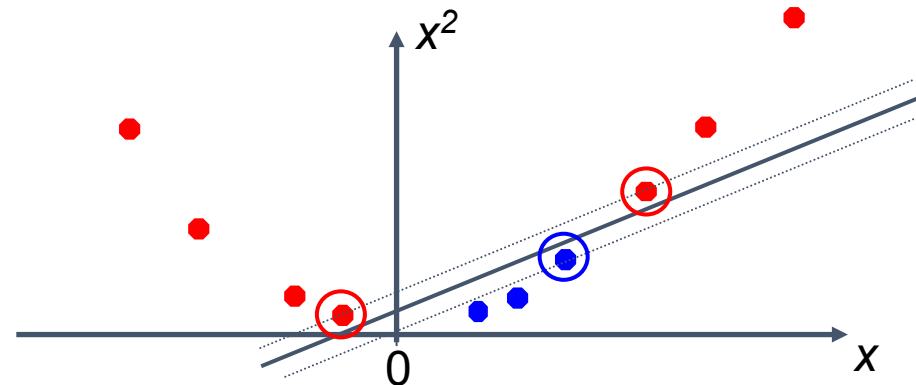


Non-linear SVMs

> Datasets that are linearly separable with some noise work out great:



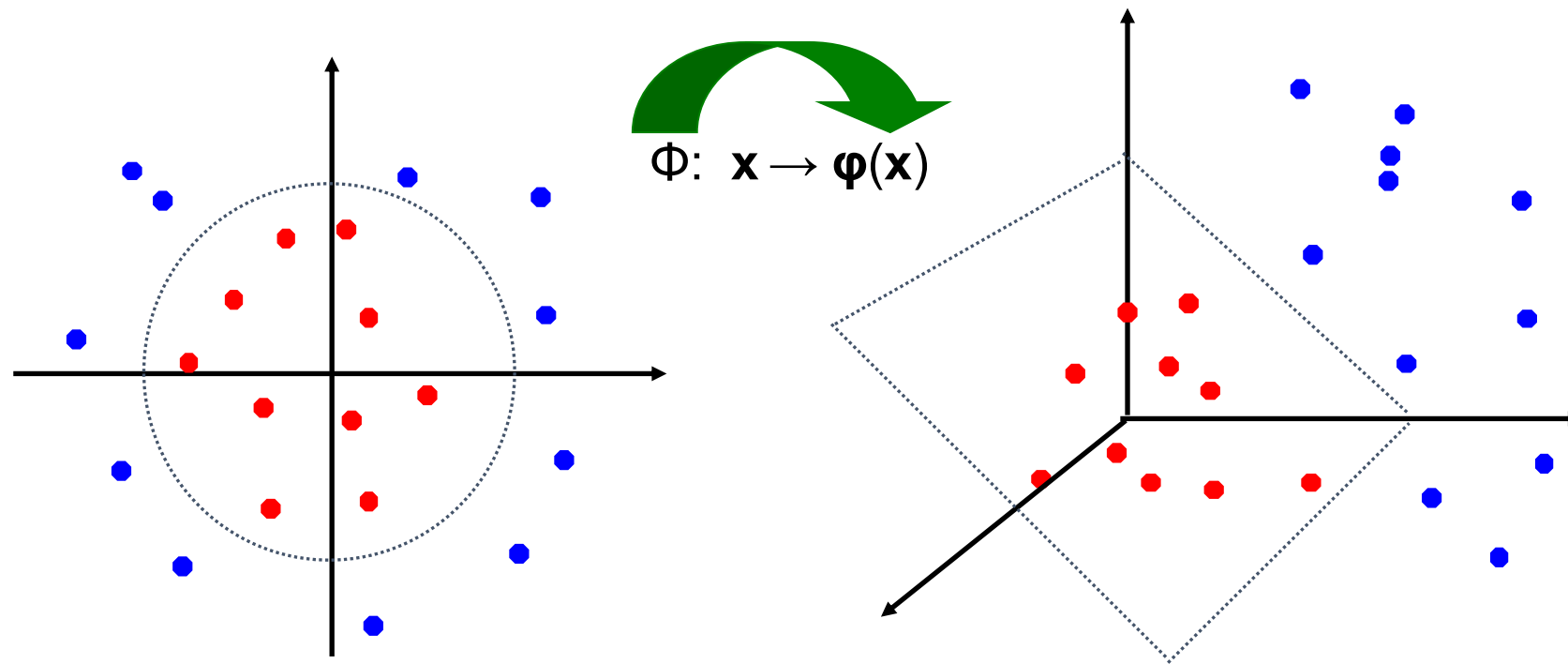
> How about... mapping data to a higher-dimensional space:





Non-linear SVMs: Feature spaces

> General idea: the original feature space can always be mapped to some higher-dimensional feature space where the training set is separable:





The “Kernel Trick”

- > The linear classifier relies on inner product between vectors $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
- > If every datapoint is mapped into high-dimensional space via some transformation $\Phi: \mathbf{x} \rightarrow \phi(\mathbf{x})$, the inner product becomes:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j) = \Phi($$

- > A *kernel function* is a function that is equivalent to an inner product in some feature space.

- > Example:

2-dimensional vectors $\mathbf{x} = [x_1 \ x_2]$; let $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^2$,

Need to show that $K(\mathbf{x}_i, \mathbf{x}_j) = \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j)$:

$$\begin{aligned} K(\mathbf{x}_i, \mathbf{x}_j) &= (1 + \mathbf{x}_i^T \mathbf{x}_j)^2 = 1 + x_{i1}^2 x_{j1}^2 + 2 x_{i1} x_{j1} x_{i2} x_{j2} + x_{i2}^2 x_{j2}^2 + 2 x_{i1} x_{j1} + 2 x_{i2} x_{j2} = \\ &= [1 \ x_{i1}^2 \ \sqrt{2} x_{i1} x_{i2} \ x_{i2}^2 \ \sqrt{2} x_{i1} \ \sqrt{2} x_{i2}]^T [1 \ x_{j1}^2 \ \sqrt{2} x_{j1} x_{j2} \ x_{j2}^2 \ \sqrt{2} x_{j1} \ \sqrt{2} x_{j2}] = \\ &= \phi(\mathbf{x}_i)^T \phi(\mathbf{x}_j), \quad \text{where } \phi(\mathbf{x}) = [1 \ x_1^2 \ \sqrt{2} x_1 x_2 \ x_2^2 \ \sqrt{2} x_1 \ \sqrt{2} x_2] \end{aligned}$$

- > Thus, a kernel function *implicitly* maps data to a high-dimensional space (without the need to compute each $\phi(\mathbf{x})$ explicitly).



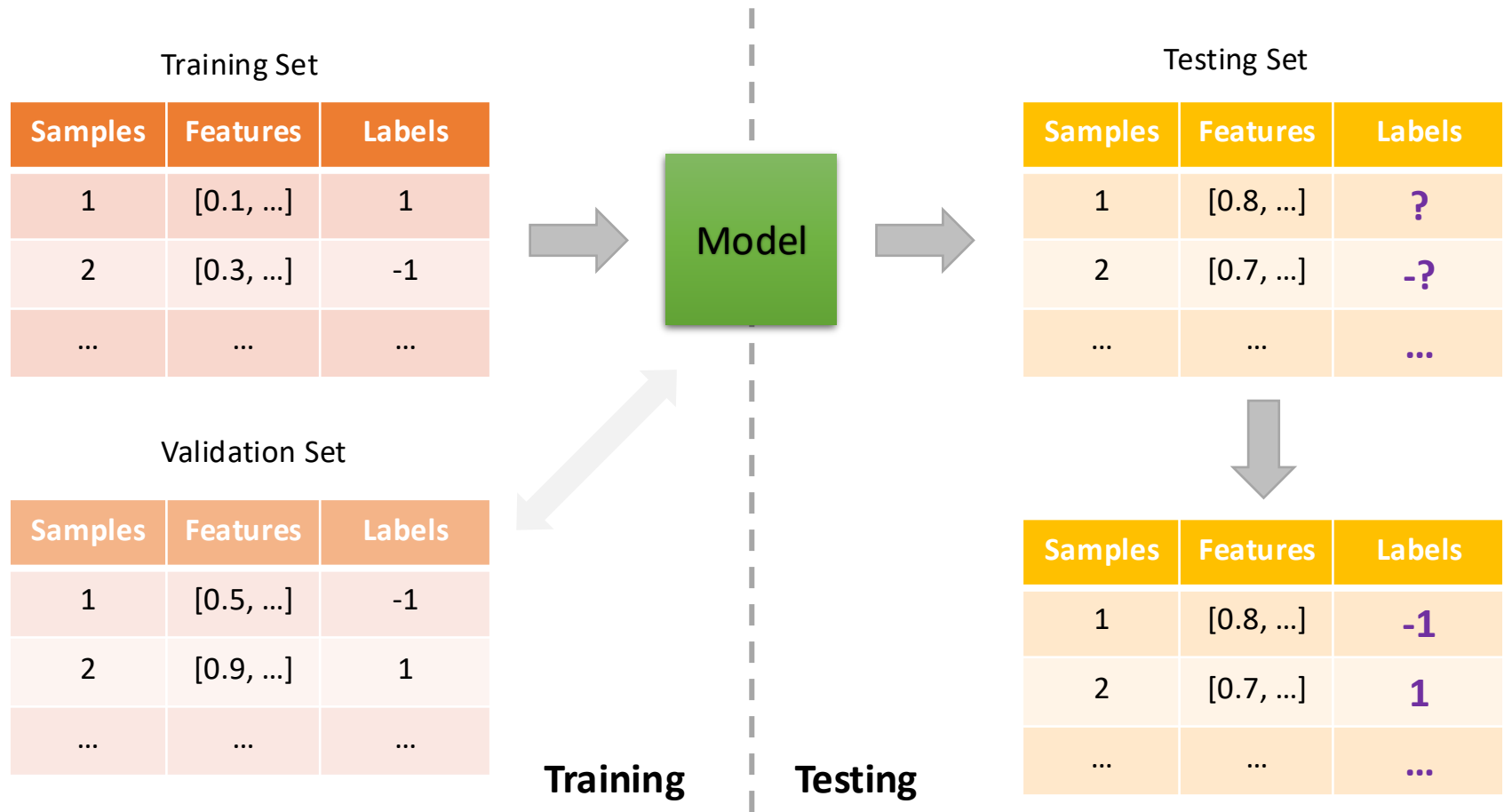
Examples of Kernel Functions

- > Linear: $K(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{x}_i^T \mathbf{x}_j$
 - Mapping $\Phi: \mathbf{x} \rightarrow \boldsymbol{\varphi}(\mathbf{x})$, where $\boldsymbol{\varphi}(\mathbf{x})$ is $\mathbf{x}$ itself
- > Polynomial of power p : $K(\mathbf{x}_i, \mathbf{x}_j) = (1 + \mathbf{x}_i^T \mathbf{x}_j)^p$
 - Mapping $\Phi: \mathbf{x} \rightarrow \boldsymbol{\varphi}(\mathbf{x})$, where $\boldsymbol{\varphi}(\mathbf{x})$ has $\binom{d+p}{p}$ dimensions
- > Gaussian (radial-basis function): $K(\mathbf{x}_i, \mathbf{x}_j) = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{2\sigma^2}}$
 - Mapping $\Phi: \mathbf{x} \rightarrow \boldsymbol{\varphi}(\mathbf{x})$, where $\boldsymbol{\varphi}(\mathbf{x})$ is *infinite-dimensional*: every point is mapped to a *function* (a Gaussian); combination of functions for support vectors is the separator.
- > Higher-dimensional space still has *intrinsic* dimensionality d (the mapping is not *onto*), but linear separators in it correspond to *non-linear* separators in original space.

RBF



Classification – Supervised Learning





The New Toy



New Toy

Testing ..

Rock

```
...n_test.py 27
ures= 300
feat= 24.7445950
954
(10)
= 2
ures= 300
feat= 24.4998880
27
(10)
= 2
ures= 300
feat= 24.7053649
932
(10)
= 2
ures= 300
feat= 24.5307696
812
(10)
= 2
ures= 300
feat= 25.1461625
038
(10)
= 2
ures= 300
feat= 26.2269585
365
(10)
= 2
ures= 300
feat= 25.9463358
177
(10)
= 2
ures= 300
```

```
144
145
if len(results) == 0:
```

Ln 138, Col 39 Spaces: 4 UTF-8 LF Python 3.10.8 64-bit



Feature Extraction



```
15 import cv2
16 import requests
17 from requests_toolbelt.multipart.encoder import MultipartEncoder
18 import numpy as np, random
19
20 cap = cv2.VideoCapture(0)
21 WindName = "Toy Program @ COMP 4423"
22 cv2.namedWindow(WindName)
23 cv2.resizeWindow(WindName, 1024, 768)
24
25 #polysmart_svr_url = 'http://127.0.0.1:8000/'
26 polysmart_svr_url = 'http://158.132.255.32:8088/'
27
28 polysmart_facerecg_svr = polysmart_svr_url+'handdetect/'
29
30 def detect(pic):
31     _, im_buf = cv2.imencode(".jpg", pic)
32     byte_im = im_buf.tobytes()
33
34     data = MultipartEncoder(fields={'file': ('img.jpg', byte_im)})
35     response = requests.post(polysmart_facerecg_svr, data=data, headers={
36         'Content-Type': data.content_type})
37     # print((response.status_code,response.json()))
38     retJson = response.json()
39     return retJson['results'] if retJson['code'] >= 0 else []
40
41 connections = [[4, 3, 2, 1, 0],# thumb
42                [8, 7, 6, 5],# index
43                [12, 11, 10, 9],# middle
44                [16, 15, 14, 13],# ring
45                [20, 19, 18, 17, 0], #pinky
46                [3, 5, 9, 13, 17]# palm
47                ]
48
```



```
50 def draw_landmarks(image, landmarks):
51     h, w, c = image.shape
52     # print([h, w, c])
53     id2cords = {}
54     for lm in landmarks:
55         idx, ftx, fty = lm['idx'], int(lm['x']*w), int(lm['y']*h)
56         id2cords[idx] = [ftx, fty]
57     for line in connections:
58         pts = [[id2cords[idx][0], id2cords[idx][1]] for idx in line]
59         pts = np.array(pts, np.int32)
60         pts = pts.reshape((-1, 1, 2))
61         image = cv2.polylines(image, [pts], False, (0, 128, 128), 4)
62     for idx in id2cords:
63         image = cv2.circle(
64             image, (id2cords[idx][0], id2cords[idx][1]), 10, (224, 224, 0), 5)
65     image = cv2.circle(
66         image, (id2cords[8][0], id2cords[8][1]), 15, (0, 0, 128), 5)
67     return image, id2cords
68
69 def extrac_feature(id2cords):
70     feat=[]
71     for id in range(21):
72         a=np.array(id2cords[id])
73         for tag in range(id+1,21):
74             b=np.array(id2cords[tag])
75             dist=np.linalg.norm(a-b)/800 # normalize the distane in the range of [0,1] by assuming the 800 is the maximum dist possible
76             feat.append(dist)
77     print('sum feat=',sum(feat))
78     return feat
79
```



```
80 feat_x,feat_y=[],[]
81 while True:
82     success, image = cap.read()
83     if not success:
84         continue
85     image = cv2.flip(image, 1)
86     imageRGB = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
87
88     results = detect(imageRGB)
89     id2cords = {}
90     #print(['len=', len(results)])
91     if len(results) == 0:
92         print("Nothing detected ...")
93     else:
94         image, id2cords = draw_landmarks(image, results[0]['landmarks'])
95
96     cv2.imshow(windName, image)
97
98     key=cv2.waitKey(1) & 0xFF
99     if key == ord('q') or key==27:
100         break
101
102     if key == ord('p') and not id2cords =={}:
103         # capture a sample for the class 'paper'
104         feat=extrac_feature(id2cords)
105         feat_x.append(feat)
106         feat_y.append(1)
107     if key == ord('r') and not id2cords =={}:
108         # capture a sample for the class 'rock'
109         feat=extrac_feature(id2cords)
110         feat_x.append(feat)
111         feat_y.append(2)
112     if key == ord('x') and not id2cords =={}:
113         # capture a sample for the class 'rock'
114         feat=extrac_feature(id2cords)
115         feat_x.append(feat)
```



Training & Testing

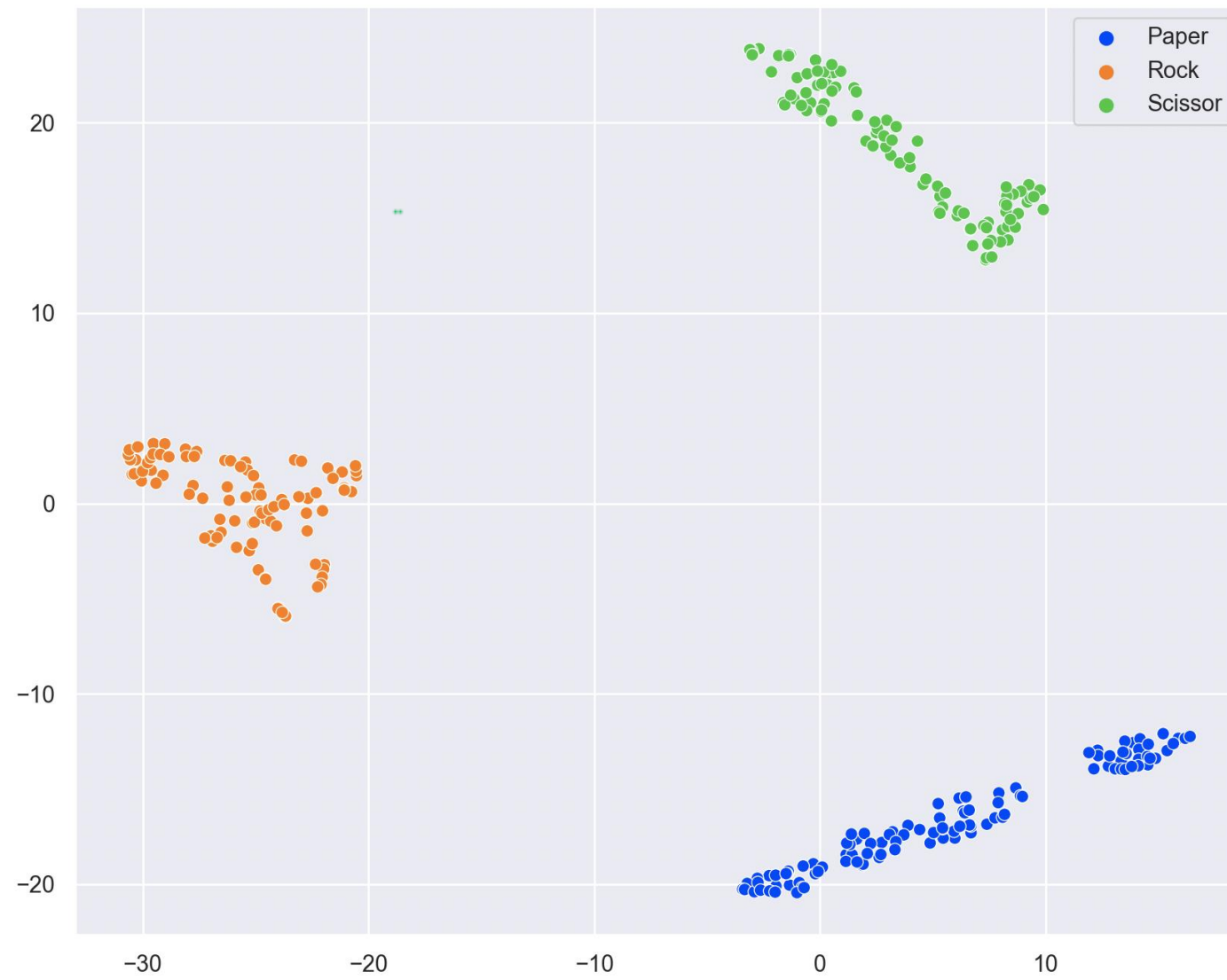




```
84 feat_x,feat_y=np.load('feat_x.npy'),np.load('feat_y.npy')
85
86 ##### run cross-validation #####
87 from sklearn import svm
88 from sklearn.model_selection import cross_val_score
89 model = svm.SVC(kernel='rbf')
90 scores = cross_val_score(model, feat_x, feat_y, cv=10)
91 print('cross validation scores: ',scores)
92 #print(feat_x.shape)
93
94 ##### visualize the data #####
95 from sklearn.manifold import TSNE
96 import seaborn as sns
97 import matplotlib.pyplot as plt
98 sns.set(rc={'figure.figsize':(10,8)})
99 palette = sns.color_palette("bright", 3)
100 tsne = TSNE()
101 X_embedded = tsne.fit_transform(feat_x)
102 label2str={1:'Paper',2:'Rock',3:'Scissor'}
103 markers=[label2str[feat_y[i]] for i in range(len(feat_y))]
104 sns.scatterplot(x=X_embedded[:,0], y=X_embedded[:,1], markers=markers, hue=markers, legend='full', palette=palette)
105 plt.show()
106
107 ##### fit the model #####
108 model.fit(feat_x,feat_y)
109
```



```
131 # show status text
132 if status_id>0: # in game mode
133     dur=time.time()-game_start_time
134     status_id=next((i for i in range(len(status_check_points)) if dur < status_check_points[i]),-1)
135     if status_id==-1:
136         status_id, game_start_time, votes = 1, time.time(), {} # restart a game
137 overlay=cv2.rectangle(overlay, (125,40),(1500,120),color=(213, 231, 242),thickness=-1)
138 image=cv2.addWeighted(overlay, 0.5, image, 0.5, 0)
139 image=cv2.putText(image,status_texts[status_id],(150,100),cv2.FONT_HERSHEY_SIMPLEX,1,(70, 62, 57),2)
140
141 id2cords = {}
142 if status_id in [0,4]:
143     results = detect(imageRGB)
144
145     if len(results) == 0:
146         print("Nothing detected ...")
147     else:
148         image, id2cords = draw_landmarks(image, results[0]['landmarks'])
149         feat=np.array([extrac_feature(id2cords)])
150         print(feat.shape)
151         label=model.predict(np.array(feat))[0]
152         print('label=',label)
153         votes[label]=votes[label]+1 if label in votes else 1
154         if status_id==0:
155             image=cv2.putText(image,label2str[label],(250,120),cv2.FONT_HERSHEY_TRIPLEX,5,(134, 152, 109),3)
156         if status_id==4:
157             computer_move=random.randint(1,3)
158
```





TODO: Study following concepts by yourself

1. <https://github.com/google-ai-edge/mediapipe>
2. Precision vs. Recall



After-class Reflection and Beyond

1. What is the main difference between image retrieval and image classification?

In image retrieval, we search for similar images. In classification, we predict labels. How are these two tasks related, and how are they different?

2. How does classification differ from clustering?

Compare classification as a supervised learning task with clustering as an unsupervised learning task.

3. What does a machine learning model learn from labeled training data?

Depending on the classifier, what could the “model” be?

4. Why do we often use a validation set in addition to a training set and testing set?

Explain the role of validation data in model selection or parameter tuning.

5. How is k-nearest neighbors related to image retrieval?

Explain how k-NN can classify an unseen image by retrieving similar labeled examples from the training set.

6. Why can $k > 1$ be more reliable than $k = 1$ in k-NN?

Explain the idea of majority voting and why it can reduce the influence of noisy examples.

7. Why does k-NN not really learn a global model of the data distribution?

Compare storing training examples with learning a probability distribution or a decision boundary.



After-class Reflection and Beyond

8. What is the “naive” assumption in Naive Bayes?

Why might assuming feature independence be problematic for image features?

9. In Bayesian classification, what is the difference between prior probability and likelihood?

Explain the meanings of $P(A)$, $P(B)$, $P(x|A)$, and $P(x|B)$.

10. Why does SVM try to maximize the margin?

Explain why a larger margin may lead to a more reliable classifier for unseen examples.

11. What are support vectors, and why are they important?

Describe why the closest training examples to the decision boundary play a central role in defining the SVM classifier.

12. What is the kernel trick, and why is it useful?

Explain how a kernel function allows SVM to work in a high-dimensional feature space without explicitly computing the mapping.



Thank You!